

**THE ONE-LINER**

An event-driven ingestion pipeline: an S3 upload fires a Lambda that routes each file by type to a streaming handler — which hashes it, stores it, and records it — with a **reconciler** to mop up partial failures.

**THE SPINE — WHAT YOU DRAW****1. Entry box — the handler signature and what fires it.**

`processUpload(key, bucket)` — the Lambda / API handler, triggered by the S3 **ObjectCreated** event.

**2. Routing — how a file type picks its handler.**

`extname(key)` → the `strategies` map (jpg, mp4, ttf, bin, zip...). An **O(1) lookup**, never a switch.

**3. The branch for an unrecognized type.**

unknown ext → `skip` (a logged no-op); a match → the format handler.

**4. Inside a handler — the single-read data flow.**

`readStream → PassThrough → [hash | s3.upload] → Promise.all`. **One disk read**, forked two ways.

**5. The export path, and its memory property.**

`cursor(batch 100) → csv → res`, the backpressure loop drawn back from socket to cursor — **constant memory** at any row count.

**6. The import path — the id-collision algorithm.**

The 4-tier ladder — **REUSE / REMAP / REGEN / INSERT** — plus the `oldId→newId` FK remap applied before each child insert.

**7. The dual-write caveat, and its fix.**

Two stores, no shared txn → track the S3 keys, **compensating-delete** on rollback. (The one people forget.)

**8. The backstop for orphans — and its one guard.**

A **reconciler** sweeps S3 for keys with no DB row → delete — but only past a grace window / **PENDING** marker, so it never touches an in-flight upload.

**9. How a redelivered event avoids double work.**

At-least-once delivery → a **processed-marker** (conditional put on the content hash). A replay sees it and no-ops — the effect is idempotent.

**DECISIONS & SWITCH CONDITIONS**

**Lambda / object** → **SQS + workers** when you need **retries, a DLQ, ordering**, burst-smoothing, or jobs over 15 min.

**During (fork)** → **After / two-pass** when the file is tiny, or an API needs the **digest before** you can store it.

**SQLite** → **JSON** when data is small, human-readable, with **no** binary or relations.

**Single PUT** → **Multipart** when large objects, flaky networks, or you want **per-part retry** and resumability.